

M11 / STUDY COMPANION

M11 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first

4

Use after learning, not instead of learning

4

DE11-01 | Row vs columnar storage; where Parquet fits

4

Core recall

4

DE11-02 | Parquet schema, row groups and metadata

4

Core recall

4

DE11-03 | Projection, predicates and analytical reads

5

Core recall

5

DE11-04 | Compression, small files and compaction

5

Core recall

5

DE11-05 | Partitioning strategy and over-partitioning

6

Core recall

6

DE11-06 | Lake vs warehouse vs lakehouse

6

Core recall

6

DE11-07 | Delta tables, transaction log and ACID concepts

7

Core recall

7

DE11-08 | MERGE/upsert and current-state records

8

Core recall

8

DE11-09 | Schema enforcement, controlled evolution and compatibility

8

Core recall

8

DE11-10 | Time travel, retention, VACUUM safety and medallion checkpoint

9

Core recall 9

Read this first

Use after learning, not instead of learning

Recall the compact card, close this book, then reproduce the mental model/control from memory. Weak recall routes to only that learner-book lesson.

DE11-01 | Row vs columnar storage; where Parquet fits

Core recall

Use Parquet when typed columnar analytical reads, compression and ecosystem interoperability matter. Keep source/raw evidence when the pipeline contract requires it; converting to Parquet is a new representation, not proof that the original source is no longer needed.

Failure to remember

The tempting slogan 'Parquet is always faster' is too broad. Reading a single tiny record, ingesting streaming bytes, or exchanging data with a simple external system can favor other formats. The engineering question is workload + access pattern + interoperability, not fashion.

Retrieval prompt

A report now needs all six columns for only two rows rather than two columns for all rows. Explain why the columnar advantage is less obvious and why file format alone cannot answer the performance question.

Visual: Data Mozart - Parquet File Format Explained 03:06-05:43 row vs column + row groups

[OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - Read and write Parquet files Columnar format, projection, schema, partitions [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/query/formats/parquet>

DE11-02 | Parquet schema, row groups and metadata

Core recall

Treat row-group statistics as observable evidence used by a reader/planner, not as a guarantee. Inspect real file metadata when diagnosing scan behavior and keep the logical predicate separate from the physical skipping opportunity.

Failure to remember

Min/max metadata is not a magical index. If each row group contains the full value range, min/max cannot exclude anything. Statistics can also be absent or affected by writer/format details. Never promise data skipping from a file extension alone.

Retrieval prompt

Change the amount predicate to amount >= 90. Re-evaluate both row groups and explain why a less selective boundary changes skipping even though the physical file did not change.

Visual: Data Mozart - Parquet File Format Explained 05:01-07:30 row groups + projection/predicate [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Apache Parquet file format Row groups, column chunks, footer metadata [OPEN SOURCE - CLICK HERE](#)

<https://parquet.apache.org/docs/file-format/>

DE11-03 | Projection, predicates and analytical reads

Core recall

Design schemas and file layout so common analytical filters have useful metadata and common projections avoid wide reads. Verify with engine evidence; SQL syntax expresses intent but does not by itself prove physical I/O.

Failure to remember

A common wrong proof is timing one tiny local query and concluding predicate pushdown works. OS cache, tiny data, engine overhead and different physical layout can dominate. Better evidence is metadata, query plan/scan metrics on a capable engine, and controlled comparisons.

Retrieval prompt

Remove the date predicate but keep only two projected columns. Explain what optimization remains and what optimization disappears.

Visual: Data Mozart - Parquet File Format Explained 05:43-07:30 projection + predicates [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - Read and write Parquet files Column projection and row-group skipping context [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/query/formats/parquet>

DE11-04 | Compression, small files and compaction

Core recall

Measure file count, typical file size, partition distribution and query behavior before tuning. Prefer platform-managed optimization where appropriate, but understand the invariant: physical rewrites must preserve logical rows/keys/totals.

Failure to remember

Do not set one universal 'perfect file size' and cargo-cult it across engines/workloads. Target sizes, automatic tuning and clustering features vary. Also do not call deleting files 'compaction' - compaction is a rewrite preserving logical data.

Retrieval prompt

Assume the table is partitioned by day and the small files are spread across 30 daily partitions. Explain why compaction may be constrained within partition boundaries and why over-partitioning can preserve the small-file problem.

Visual: Data Mozart - Parquet File Format Explained 07:30-10:04 file size + compression [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - Optimize data file layout Compaction/file-layout maintenance [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/tables/operations/optimize>

DE11-05 | Partitioning strategy and over-partitioning

Core recall

Partitioning is a physical optimization contract. Keep partition cardinality and files-per-partition observable; do not let an old layout silently become a new bottleneck as volume and query patterns change.

Failure to remember

Do not choose a partition key only because users filter it sometimes. Estimate distinct values, data per partition, filter selectivity, write pattern and maintenance behavior. A high-cardinality timestamp can create vast numbers of tiny directories/files.

Retrieval prompt

The table grows from 50 GB to 5 TB and common filters move from region to date ranges. Explain why a physical-layout decision should be revisited instead of treated as permanent truth.

Optional visual: Data Mozart - Parquet File Format Explained 05:01-10:04 row groups/projection/file-size context; this video does NOT teach partition strategy. Official partition docs + book are required. [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - When to partition tables Partitioning vs over-partitioning; current alternatives [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/tables/partitions>

DE11-06 | Lake vs warehouse vs lakehouse

Core recall

Choose architecture by guarantees, workloads, interoperability, cost and operating skill. Keep raw evidence, curated contracts and serving models distinct even when one platform exposes them through a unified UI.

Failure to remember

Do not claim 'lakehouse replaces every warehouse' or 'lake means ungoverned mess.'
Governance and modeling depend on implementation. Also do not call a folder of Parquet files a transactional table merely because a SQL engine can query them.

Retrieval prompt

Assume regulations require immutable raw evidence and strong audited serving tables. Explain how a hybrid architecture might intentionally keep both raw lake storage and governed warehouse/lakehouse tables.

Optional visual: Data Mozart - Parquet File Format Explained 10:04-11:28 Delta-format introduction only; lakehouse architecture is taught by the book + current official docs. [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - What is a data lakehouse? Lake vs warehouse vs lakehouse role [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/lakehouse/>

DE11-07 | Delta tables, transaction log and ACID concepts

Core recall

Treat the transaction log as table metadata/state, not optional bookkeeping. Use supported table APIs/commands so file rewrites, metadata and versions stay coherent.

Failure to remember

Never edit Parquet files or `_delta_log` internals by hand as a normal repair strategy. A table format's guarantees depend on coordinated log + data-file semantics. Random file deletion can corrupt the table or destroy time-travel capability.

Retrieval prompt

Two writers start from version 1 and both want to update the same business keys. Explain why a transaction protocol needs conflict detection/validation instead of letting both silently overwrite directory contents.

Optional visual: Databricks - Diving into Delta Lake: Transaction Log No independently verified bounded chapter range assigned; book + current official Delta docs are required. [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=IXlBjdehsdw>

Official: Databricks - What is Delta Lake? Transaction log, ACID, schema enforcement, metadata [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/delta>

DE11-08 | MERGE/upsert and current-state records

Core recall

Make MERGE deterministic at the source boundary and idempotent for reruns. Reconcile affected keys/counts/totals and keep change-event evidence when the business requires history.

Failure to remember

MERGE is not a license to skip source-grain checks. A source with duplicate business keys can be ambiguous and may error or produce wrong current-state logic depending on engine/rules.

DISTINCT is not a universal repair because two different updates can remain genuinely conflicting.

Retrieval prompt

Add a delete event to the source. Explain why current-state deletion policy and historical/audit requirements must be decided before adding a WHEN MATCHED . . . DELETE branch.

Optional visual: Databricks - Deep-Dive into Delta Lake No independently verified bounded chapter range assigned; MERGE/source-key safety comes from the book + current official docs.

[OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=de-6a6Bfw6E>

Official: Databricks - Upsert with MERGE Matched/not-matched update/insert patterns [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/delta/merge>

DE11-09 | Schema enforcement, controlled evolution and compatibility

Core recall

Version schema contracts. Prefer explicit, reviewed evolution for important tables. Test downstream compatibility and keep rescue/quarantine paths for unexpected producer changes rather than silently coercing them into trusted data.

Failure to remember

A new nullable column can be low-risk; renaming a field used by downstream jobs is not. Some platforms support automatic evolution for specific change classes, but support does not mean the business meaning is safe. Schema compatibility is both technical and contractual.

Retrieval prompt

Assume a BI report selects columns by name and a streaming reader keeps a tracked schema. Explain why rename/drop planning must include consumers and not only the writer that evolves the table.

Optional visual: Databricks - Deep-Dive into Delta Lake No independently verified bounded chapter range assigned; schema/evolution behavior comes from the book + current official docs.

[OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=de-6a6Bfw6E>

Official: Databricks - Schema evolution Supported change classes and explicit evolution controls
[OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/data-engineering/schema-evolution>

DE11-10 | Time travel, retention, VACUUM safety and medallion checkpoint

Core recall

Make retention a business/recovery policy, not a random storage tweak. Use table-supported maintenance commands, preserve evidence of changes, and connect Bronze/Silver/Gold layers with explicit contracts and reconciliation rather than names alone.

Failure to remember

Do not treat VACUUM as a performance command that can be run with arbitrarily tiny retention. Removing old files can permanently eliminate time-travel/recovery capability and can be dangerous for long-running jobs. Also distinguish OPTIMIZE (rewrite layout) from VACUUM (remove eligible unused files).

Retrieval prompt

Finance requires 30 days of reproducible month-close recovery instead of 7. Explain which retention decision changes and why the storage-cost trade-off is intentional rather than an error.

Optional visual: Databricks - Diving into Delta Lake: Transaction Log No independently verified bounded chapter range assigned; time-travel/history/VACUUM behavior comes from current official docs + book. [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=IXlBJdehsdw>

Official: Databricks - Table history Versions and history [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/tables/history>